

Предложения по улучшению подпроекта main

Проект fullWeb (Query) - модуль main: Kotlin/JS Frontend (React + MUI + ag-grid-community). Дата: 20.08.2026

Исходный анализ

Файлы модуля: Main.kt (вход, регистрация ag-grid), AppFC.kt (главный компонент AppClients: меню, панель кнопок, диалоги, grid), AppFCFun.kt (логика: построение колонок, fetch, история, даты), ApiQueryys.kt (HTTP-клиент ktor, парсинг URL), FC.kt (InputFC, FieldDateFC), interop-обёртки (ag-grid, mui-nested-menu, react-share), константы и helper-функции в shared/CommonVars.kt.

Основные проблемы: глобальное mutable-состояние на уровне top-level var, хак useStateMap/useEffectMap (setState складываются в глобальную map), необработанные краевые случаи (пустой результат, битые даты), блокирующие window.alert, много мёртвого/закомментированного кода.

Фаза 1 - Быстрые баг-фиксы (риск: низкий)

Каждый пункт - отдельный коммит. Проверка: сборка :main:jsDevelopmentExecutable + ручной прогон в браузере (CDP, вкладка Queryys).

#	Проблема	Файл	Решение
1	Ведущий пробел в URL при сохранении	ApiQueryys.kt:23	Убрать пробел в начале строки post(...)
2	Краш при пустом результате: JSON.stringify(rows[0]) при rows.isEmpty()	AppFCFun.kt arrayOfColumnDefs:165, getColumns:422	Guard: if (rows.isEmpty()) return arrayOf()
3	strToDate бросает исключение на пустых/битых значениях - падение сортировки дат	AppFCFun.kt:299-323	Fallback-дата вместо throw + null-safe dateComparator
4	loadByPost может зависнуть навсегда при endpoint==null	ApiQueryys.kt:48-62	Убрать suspendCancellableCoroutine: endpoint?.let{...} ?: error(...)
5	gridApi!! - NPE при кликах до onGridReady	AppFC.kt:479, AppFCFun.kt:61,151,380	Null-safe gridApi?.let{...} + ранний return
6	Удаление строк без подтверждения	AppFCFun.kt:151-161 deleteRows	window.confirm перед пометкой на удаление
7	Кнопка сору делает повторный сетевой запрос fetchArray(urlInput)	AppFC.kt:369-383	Клонировать данные из gridApi.getSelectedRows()

Фаза 2 - UX-улучшения (риск: средний)

1. window.alert -> MUI Snackbar/Alert: viewResult, fetchArray.onFailure, ошибка меню, jsr223, lunchQuery NOT_FOUND. Блокирующие алерты заменяются ненавязчивыми сообщениями.
2. Индикатор загрузки: "Trying to fetch/save..." -> MUI CircularProgress / LinearProgress (info(), AppFC.kt:144-160).
3. Замена img-кнопок (12 шт.) на IconButton + @mui/icons-material: refresh, help, share, prev/history/next, interval, edit, delete, copy, save. Уйдут PNG из resources/img.
4. Доступность: aria-label/alt у кнопок; меню на hover не работает на тач-устройствах - добавить клик-раскрытие.
5. История запросов в localStorage (queryHistory) - переживает перезагрузку; лимит 50 записей; кнопка очистки.
6. dateFrom/dateTo в URL (dFrom/dTo) - поделиться ссылкой с интервалом дат.

Фаза 3 - Архитектура (риск: высокий; отдельными шагами с регрессией)

Шаг 3.1. Группировка глобального состояния

Топ-уровень var (AppFC.kt, AppFCFun.kt, CommonVars.kt): scope, urlInput, urlQuerys, contextGreed, queryCurrent, NewValue, currentPlaceholder, adminState, v_querys, endpoint_service_url, useEffectMap, rowFieldsValueString, dateFrom, dateTo, queriesLoaded, queryHistory, historyCurrent, gridApi, startEnv, rowFieldsValueMap. Сгруппировать в data class AppState + AppStateHolder. Поведение не меняется, состояние локализуется.

Шаг 3.2. Удаление хака useStateMap/useEffectMap

useEffectMap + setStateMap (AppFCFun.kt:461-468, AppFC.kt:56,75-81) заменить явной передачей setter-функций (setRowList, setColList, setResultUpdate, setShowButton, setEditIcon, setTitleCurrent) в функции-хелперы. Самый важный рефакторинг.

Шаг 3.3. Внешний store (опционально)

zustand или самописный store на корутинах: AppStateStore со stateFlow, подписка через useState + LaunchedEffect. Убирает MainScope и почти все top-level var.

Шаг 3.4. Типизация

@Serializable data class RowData; типизированный builder колонок вместо "магической" строки в getColumns; ParamsAG/ColDefAG - конкретные типы вместо dynamic.

Шаг 3.5. Убрать побочный эффект var dateFormat

AppFCFun.kt:15,183: dateFormat устанавливается в arrayOfColumnDefs - передавать формат в dateComparator параметром/closure.

Шаг 3.6. Вычистить мёртвый код

Закомментированные блоки: AppFC.kt:52,474-478,492-498 (enableRangeSelection, onSortChanged, sorted), ReactGrid.kt (editType, theme), CommonVars.kt:19,57-74 (старый decodeJson/encodeJson), AppFCFun.kt:470-478 (cellMouseOver), brython в index.html. Range-selection: включить или удалить (отдельный коммит).

Шаг 3.7. Собственное меню вместо mui-nested-menu

Рекурсивный компонент на базе MUI Menu/Popover + onMouseEnter/onClick (AppFC.kt:162-191,406-426; muiNestedMenu.kt). Убирает устаревшую зависимость и даёт полный контроль.

Фаза 4 - Качество и инфраструктура (инкрементально)

1. Юнит-тесты (main/src/jsTest): strToDate, dateComparator, getColumns, arrayOfColumnDefs, getParameter, insertParameterValue, nameValue. kotlin.test + ktor-client-mock. Команда: gradlew :main:jsTest.
2. Линтер ktlint (org.jlleitschuh.gradle.ktlint) в build.gradle.kts; :main:check включает линт.
3. CI: GitHub Actions - сборка :main:jsProductionExecutable + jsTest + check (Java 25).
4. Оптимизация бандла ag-grid: вместо AllCommunityModule - ClientSideRowModelModule, TextFilterModule, NumberFilterModule, DateFilterModule, UndoRedoEditModule. Замерить размер до/после.
5. Серверная пагинация (ServerSideRowModelModule + backend LIMIT/OFFSET) - стратегическая задача для больших таблиц.
6. Google Fonts: убрать блокирующий @import в menu.css, использовать preconnect + media print.
7. help.html: перевести на русский, добавить параметр rFV и новые хоткеи; единый язык интерфейса.

Порядок выполнения

Шаг	Оценка	Риск	Точка регрессии
Фаза 1 (7 фиксов)	0.5-1 день	низкий	edit/save/delete, пустые таблицы
Фаза 2 (UX)	1-2 дня	средний	панель кнопок, история, интервал
Шаг 3.1-3.2	2-3 дня	высокий	всё: меню, грид, редактирование, история
Шаг 3.3-3.5	2-3 дня	высокий	навигация, гонки состояний, колонки
Шаг 3.6-3.7	1-2 дня	средний	меню, hotkeys
Фаза 4	инкрементально	низкий	сборка, CI

Замечания

- Фазу 3 нельзя смешивать с Фазой 1-2 - они затрагивают одни и те же строки.
- После каждого шага Фазы 3 - полный smoke-тест в браузере (меню через cdp-open-menu.ps1, edit/save/delete, история, share-ссылка, интервал дат).
- Перед началом каждой фазы зафиксировать текущее поведение скриншотами (ScreenNNN.png по протоколу .clinerules) для сравнения до/после.